

MODULE 5 - WIRE UP

MCP Connector Cheat Sheet

Which connector for which tool. Auth notes you can trust.

Version: v1 - 2026-05-09

Author: Chase Whited / Whited Consulting

Stable URL: raw.githubusercontent.com/cwhited26/aipocketagency-brain

MCP Connector Cheat Sheet

MCP (Model Context Protocol) connectors give agents structured access to external systems - databases, APIs, file stores, calendars, payment processors. Picking the right connector for the job is half the work. This cheat sheet is the working table.

Pick by task

You want to...	Connector	Auth notes
Run SQL against your hosted Postgres	Supabase MCP	Service-role key for server-side; anon key + RLS for user-context. Never expose service-role to a client agent.
Search and read GitHub repos, files, issues, PRs	GitHub MCP	Personal access token with repo scope; read-only when possible.
Read or send Slack messages	Slack MCP	OAuth app install in your workspace; user token; scope to the channels the agent should reach.
Read or send email	Gmail / Outlook MCP	OAuth flow; scope to read or send (request least privilege).
Manage calendar events	Google Calendar / Calendly / Cal.com MCP	OAuth or API key per provider; primary calendar only unless multi-calendar is needed.
Run payments / read invoices	Stripe MCP	Restricted API key. Never the secret key with full scope. Read-only for analytics agents.
Search the live web	Bright Data / search MCP	API key per provider; rate-limit aware.
Read or write Notion / Confluence pages	Provider-specific MCP	OAuth or integration token; scope per workspace.
Trigger a CI/CD pipeline	GitHub Actions MCP / provider MCP	Token with workflow:write scope; never personal admin tokens.
Read your hosted brain	Custom: REST -> /v1/brain/*	Bearer token from the dashboard. Scope: read or write. Audit log captures every call.

Auth defaults that hold up

- **Least privilege.** Read-only when possible. Scoped to one project or one workspace.
- **Per-agent keys.** One key per agent, named after the agent. Revoke when the agent retires.
- **Env var names not values.** Reference \$STRIPE_RESTRICTED_KEY in docs and skills - never the value.

- **Audit log.** Every connector that supports an audit trail should be enabled. Your brain logs every write; demand the same of upstream tools.
- **Rotate quarterly.** Mark a calendar reminder. Old keys leak from old places.

When NOT to use a connector

If the task is one-off, manual, and you trust yourself to do it once, skip the connector. Connectors are infrastructure - they are worth their setup cost only when the workflow recurs.

If the task is high-trust (sending money, executing trades, deleting customer data), do not give an agent the connector. Have the agent draft the action; you execute it manually.

Wiring a new connector - the standard sequence

- 1. Create the credentials in the upstream system. Scope to the minimum required.
- 2. Save the credential to an env var. Reference it by name in the connector config.
- 3. Test with a read-only call from the connector before wiring write access.
- 4. Add an entry to memory-types/reference.md - what the connector is and what it can do.
- 5. Add an entry to memory-types/project.md - which project uses it and the scope locked.
- 6. Document any rate limits, quirks, or gotchas in a note inside the connector entry. Future you will thank you.

The right number of connectors is the smallest number that covers your recurring workflows. Adding a connector is easy; getting rid of an unused one is awkward.